Tarea 02

Space Invaders

Profes: Nancy Hitschfeld Kahler, Valentín Muñoz Apablaza
Auxiliares: Julieta Coloma, Catalina Gajardo

1 - Introducción

En esta tarea tendrán que programar un mini-juego similar a *Space Invaders*.

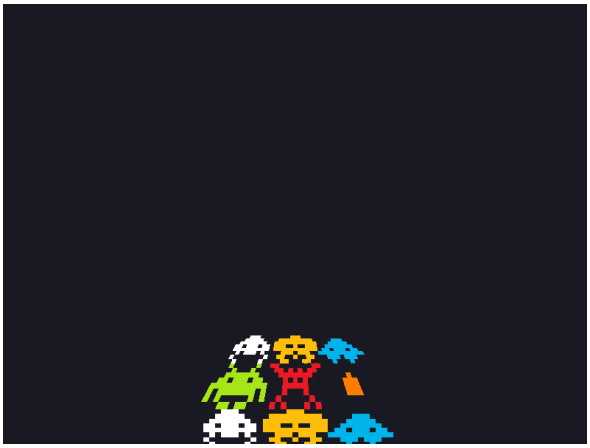


Figura 1: Screenshot de una posible solución a la tarea, también vean este video

1 -1 - Template

Junto a este enunciado se les entrega una carpeta que contiene el esqueleto de su proyecto.

La mayor parte del trabajo la deben realizar en el módulo `invaders.py` donde completarán las funciones. Pero también deben completar el programa principal `tarea2.py` y los shaders. Está claramente indicado con **comentarios** donde falta completar en los archivos.



Cuando ejecuten el template sin cambios deberían ver algo como lo siguiente, donde se muestra un objeto con una textura.

Cuando terminen su tarea simplemente vuelvan a comprimir esta carpeta completa y la entregan tan cual :)

2 - Trabajo a realizar

A continuación se describe todo lo que debe incluir su tarea.

2 -1 - Las cámaras (0.5pt)

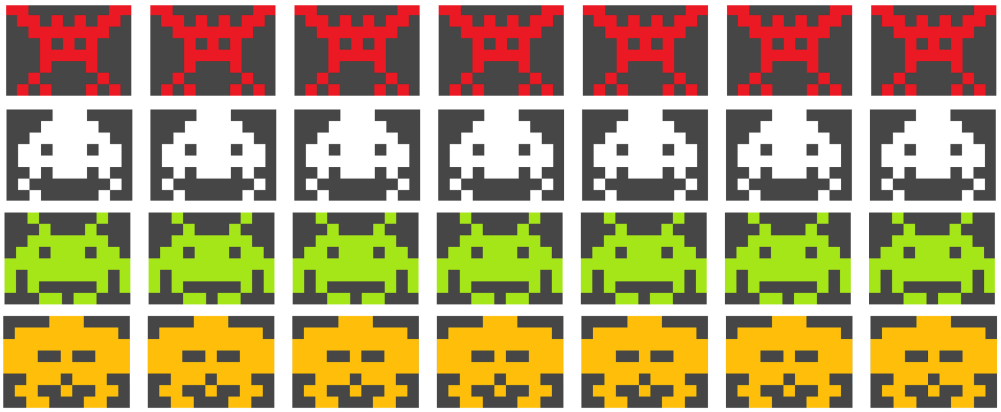
Se debe contar con dos cámaras, y al presionar la tecla `C` se debe cambiar entre ellas. La primera cámara seguirá a la nave en su posición, mirando hacia los enemigos. La segunda cámara se encontrará sobre todos los objetos y será estática, permitirá observar toda la escena desde arriba. Pueden ver ejemplos de ambas cámaras en el video de ejemplo.

2 -2 - La nave (0.5pt)

La nave corresponderá a un objeto con una textura. Este objeto puede ser tan simple como un cubo o tan complejo como usted quiera. La nave podrá moverse **solo** en el eje X, entre -6 y 6, controlada por el teclado. Se le sugiere leer bien el template para hacer uso de lo que ya viene dado ;).

2 -3 - Los enemigos (1.1pt)

Los enemigos también serán objetos con texturas. Estos vienen en grupos y se organizan en filas y columnas en los ejes X y Z. Cada grupo de enemigos comparte una misma textura, y cada grupo se encuentra en su propia fila:



Los enemigos se mueven todos en conjunto, hacia la izquierda o la derecha entre $X \in [-10, 10]$. Cuando un enemigo llega a un límite, -10 o 10 en X, se mueven hacia la nave en el eje Z y cambian su dirección de movimiento. Para entender mejor este movimiento, vean el video de ejemplo!

Deben haber **al menos 4 grupos de enemigos** distintos.

2 -4 - Balas (1.4pt)

La nave disparará balas cuando se presione la tecla SPACE. Esto implica que se crea un **objeto texturizado** en la posición actual de la nave y se mueve con velocidad constante en el eje Z.

Cuando una bala colisiona con un enemigo morir y el enemigo también. Además cuando una bala no logra colisionar con un enemigo y llega al final de la pantalla, debe desaparecer.

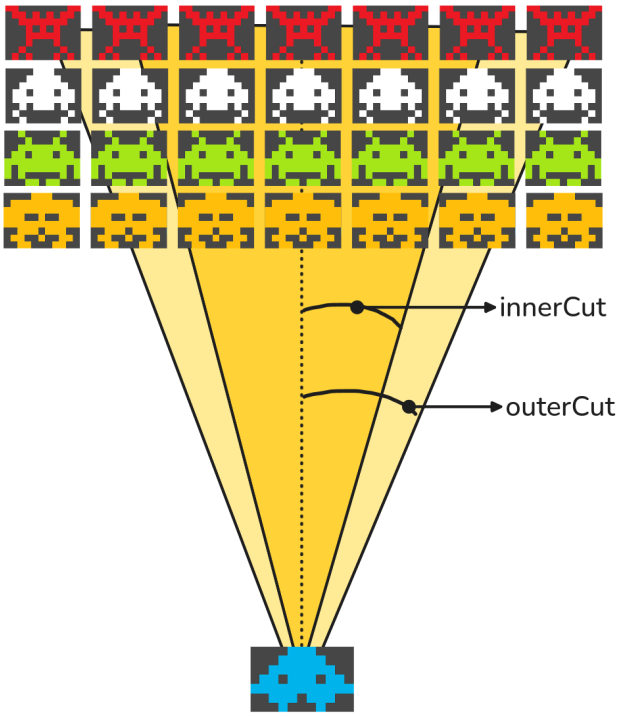
En el template se proporciona una función que les ayudará a detectar la colisión entre una bala y un enemigo.

2 -5 - Iluminación (2.5pts)

2 -5 -1 - Efecto Foco

Podrán ver en el video de ejemplo, que hay un *foco de iluminación* saliendo de la nave. Tendrán que implementar esta iluminación en el fragment shader y pasar los uniforms necesarios para lograrlo. Este efecto se logra de la siguiente forma:

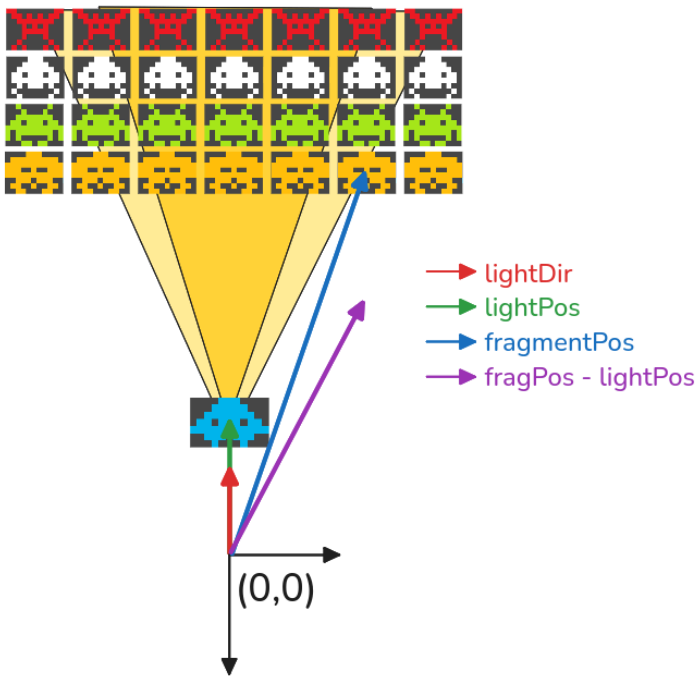
1. Se quieren definir dos constantes para los ángulos de corte del foco:



2. Luego, se tiene que como la luz sale de la nave y va en dirección de los enemigos, su posición y dirección serán:

```c
vec3 ligthPos = shipPos;
vec3 lightDir = vec3(0, 0, -1);
```

3. Luego para saber si un objeto esta en la oscuridad o iluminado, debemos saber que tan cerca está del centro del foco de la luz. Esto es equivalente a la dirección de la luz. Para comparar qué tan cerca están dos vectores, se puede utilizar el producto punto. En este ejemplo, nos interesaría saber qué tan cerca están los vectores rojos y morados.



4. Ese producto nos dará un `float`. Tendremos que comparar ese valor con los de innerCut y outerCut, para saber si está **suficientemente** cerca. Se recomienda usar la siguiente función para lograr un resultado fluído:

```c
// si angle es menor a 0.94 -> intensidad = 0
// si angle es mayor a 0.985 -> intensidad = 1
// aquí la función smoothstep es perfecto pq, dado un min, un max y un value,
// si value es <= min, entrega 0,
// si value es >= max, entrega 1,
/// si está en medio entrega un valor entre 0 y 1
float intensity = smoothstep(
    outerCone,
    innerCone,
    angle
);
```

2 -5 -2 - Iluminación General

Además del *efecto foco*, se le pide agregar la iluminación ambiental y difusa de una **luz direccional** que mira hacia abajo, es decir con dirección $(0, -1, 0)$. Esto se puede realizar de la misma forma que fue visto en clases y solo depende de la normal y la posición del fragmento. Está indicado en el template donde debe agregar este componente.

2 -6 - Funciones específicas

Para lograr lo anterior, **debe** completar las siguientes funciones:

- `init_enemies(world, controler)`: Esta función no entrega nada pero agrega a los enemigos en sus posiciones iniciales al grafo de escena y al controlador.
- `shoot(ship_position, world, bullet_mesh, controller)`: Esta función es llamada cuando el usuario presiona SPACE (vea el archivo principal para corroborarlo). Esta función debe crear una bala en la posición de la cámara y agregarla al grafo de escena y el controlador.

- `update_ship(world, controller)`: Esta función se llamará en cada frame y debe actualizar la posición de la nave en el grafo de escena, de acuerdo al input del usuario.
- `update_enemies(world, controller)`: Esta función se llamará en cada frame y debe actualizar la posición de los enemigos vivos en el grafo de escena. Debe reproducir el movimiento explicado en el enunciado.
- `update_bullets(world, controller)`: Esta función se llamará en cada frame y actualiza la posición y el estado de las balas vivas en el grafo de escena. Además revisa si una bala ha colisionado con un enemigo, actualizando la bala y el enemigo.
- `update_camera(ship_camera, dt, world)`: Esta función se llama en cada frame y actualiza la posición de la cámara en función de la posición de la nave.

Además debe completar el archivo principal `tarea2.py` y el shader `tarea2.frag`.

2 -7 - Material Complementario

- **Auxiliares**: Vean el material entregado en el repositorio tanto de este año como años anteriores.
- **Template de la tarea**:

3 - Bonus

Se otorgará un bono de hasta 0.5 puntos adicionales en alguna tarea (acumulable para las futuras) si programa **uno** de los siguientes elementos:

- Implemente la posibilidad de pausar el juego, es decir, que al presionar alguna tecla el movimiento de los enemigos se detiene, la nave deja de poder moverse y no se pueden lanzar balas.
- Implemente la posibilidad de apagar y prender el foco de la nave.

4 - Entregable

Se debe adjuntar un archivo .zip que contenga todos los elementos necesarios para ejecutar la tarea correctamente. Esto incluye, imágenes requeridas, módulos utilizados, etc.

Se aplicarán descuentos en la calificación en los siguientes casos:

- −0.3 puntos por nombres mal escritos.
- −0.3 puntos por errores en las rutas o caminos de los archivos.
- −0.5 puntos por el uso de librerías no incluidas en el curso sin previo aviso.
- −0.5 puntos por la entrega con archivos faltantes.

Si su tarea no puede ser ejecutada por alguna otra razón quedará a criterio del equipo docente evaluarla con la posibilidad de nota mínima si es que lo amerita.

Recomendaciones:

1. Descargar el archivo `.zip` y verificar que el programa se ejecuta debidamente.
2. Incluir un archivo `README.txt` en caso de que la ejecución de la tarea requiera pasos adicionales.

5 - Consideraciones

Debe tener en cuenta lo siguiente:

- El plazo de entrega es inamovible.
- El trabajo es individual.
- No está permitido el plagio del trabajo de sus compañeros.
- Es obligatoria la entrega de la declaración de uso de IA.